

# CRRSS Usage Guide

---

## Table of Contents

---

1. [Getting Started](#)
2. [Command-Line Usage](#)
3. [Programmatic Usage](#)
4. [Configuration](#)
5. [Best Practices](#)
6. [Troubleshooting](#)

## Getting Started

---

### Installation

After building CRRSS, you can use it in two ways:

1. **From build directory:** `./build/crrss <command>`
2. **After installation:** `crrss <command>` (if installed system-wide)

### Quick Start

```
# Check version
crrss version

# Get help
crrss help

# Analyze a file
crrss query --file mycode.c --details

# Get statistics
crrss stats
```

## Command-Line Usage

---

### Query Command

The `query` command searches for bug predictions and risk assessments.

### Basic Usage

```
# Query all high-priority bugs
crrss query --priority P1

# Query memory-related issues
crrss query --category memory

# Query specific file
crrss query --file kernel/memory.c
```

## Advanced Usage

```
# Combine filters
crrss query --priority P0 --category security --details

# Limit results
crrss query --priority P1 --max-results 20

# Query with detailed output
crrss query --file scheduler.c --details
```

## Query Options

| Option            | Description               | Values                                            |
|-------------------|---------------------------|---------------------------------------------------|
| -p, --priority    | Filter by bug priority    | P0, P1, P2, P3                                    |
| -c, --category    | Filter by bug category    | memory, concurrency, logic, performance, security |
| -f, --file        | Analyze specific file     | file path                                         |
| -d, --details     | Show detailed information | flag                                              |
| -n, --max-results | Maximum results           | number                                            |

## Priority Levels

- **P0 (Critical):** System crash, data corruption, security vulnerabilities
- **P1 (High):** Major functionality broken, memory leaks, race conditions
- **P2 (Medium):** Minor functionality issues, code quality problems
- **P3 (Low):** Cosmetic issues, optimization opportunities

## Bug Categories

- **Memory:** Memory leaks, use-after-free, buffer overflows
- **Concurrency:** Race conditions, deadlocks, synchronization issues
- **Logic:** Logic errors, incorrect algorithms
- **Performance:** Performance bottlenecks, inefficient code
- **Security:** Security vulnerabilities, unsafe operations

## Stats Command

The `stats` command displays codebase statistics and system health metrics.

## Basic Usage

```
# Show all statistics
crrss stats

# Show statistics for directory
crrss stats --directory moduler_kernel/

# Show memory statistics
crrss stats --memory
```

## Advanced Usage

```
# Detailed statistics with memory info
crrss stats --directory . --memory --validation

# Export to JSON
crrss stats --format json > stats.json

# Export to CSV
crrss stats --format csv > stats.csv
```

## Stats Options

| Option           | Description                | Values          |
|------------------|----------------------------|-----------------|
| -d, --directory  | Analyze directory          | directory path  |
| -m, --memory     | Show memory statistics     | flag            |
| -v, --validation | Show validation statistics | flag            |
| --format         | Output format              | text, json, csv |

## Output Interpretation

### Bug Prior Mapping Engine Statistics:

- Total Scans: Number of files analyzed
- Bugs Predicted: Total potential bugs found
- Prediction Accuracy: Confidence in predictions

### Self-Check Internal Validator Statistics:

- Total Validations: Number of files validated
- Total Issues: Number of validation issues
- Average Compliance: Code quality score (0-100%)

### Memory Integration Statistics:

- Total Allocations: Total memory allocated
- Total Frees: Total memory freed
- Current Usage: Currently allocated memory
- Memory Efficiency: Ratio of freed to allocated memory

## Programmatic Usage

### Using BPME

```
#include "bpme/bpme.h"

int main() {
    // Configure BPME
    bpme_config_t config = {
        .knowledge_base_path = NULL,
        .enable_ml_predictions = false,
        .enable_pattern_matching = true,
        .confidence_threshold = 0.7,
        .max_predictions = 100
    };

    // Initialize
    bpme_context_t* ctx = bpme_initialize(&config);
    if (!ctx) {
        fprintf(stderr, "Failed to initialize BPME\n");
        return 1;
    }

    // Analyze file
    bug_prediction_t predictions[100];
    uint32_t num_predictions = 0;

    crrss_status_t status = bpme_analyze_file(
        ctx, "myfile.c", predictions, 100, &num_predictions
    );

    if (status == CRRSS_SUCCESS) {
        printf("Found %u potential issues:\n", num_predictions);

        for (uint32_t i = 0; i < num_predictions; i++) {
            printf("\n[%s] %s:%u\n",
                bug_priority_to_string(predictions[i].priority),
                predictions[i].file_path,
                predictions[i].line_number);
            printf("  %s\n", predictions[i].description);
            printf("  Recommendation: %s\n", predictions[i].recommendation);
        }
    }

    // Cleanup
    bpme_shutdown(ctx);
    return 0;
}
```

## Using SCIV

```
#include "sciv/sciv.h"

int main() {
    // Configure SCIV
    sciv_config_t config = {
        .enable_strict_mode = true,
        .enable_style_checks = true,
        .enable_performance_checks = true,
        .max_function_complexity = 20,
        .max_function_lines = 200,
        .max_cyclomatic_complexity = 15,
        .coding_standard = "kernel"
    };

    // Initialize
    sciv_context_t* ctx = sciv_initialize(&config);
    if (!ctx) {
        fprintf(stderr, "Failed to initialize SCIV\n");
        return 1;
    }

    // Validate file
    validation_issue_t issues[100];
    uint32_t num_issues = 0;

    crrss_status_t status = sciv_validate_file(
        ctx, "myfile.c", issues, 100, &num_issues
    );

    if (status == CRRSS_SUCCESS) {
        printf("Found %u validation issues:\n", num_issues);

        for (uint32_t i = 0; i < num_issues; i++) {
            printf("\n[%s] %s:%u - %s\n",
                validation_result_to_string(issues[i].result),
                issues[i].file_path,
                issues[i].line_number,
                issues[i].rule_name);
            printf("  %s\n", issues[i].message);
            printf("  Suggestion: %s\n", issues[i].suggestion);

            // Free allocated strings
            free((void*)issues[i].file_path);
            free((void*)issues[i].message);
            free((void*)issues[i].suggestion);
        }
    }

    // Cleanup
    sciv_shutdown(ctx);
    return 0;
}
```

## Using Memory Integration

```
#include "memory_layer/memory_integration.h"

int main() {
    // Configure Memory Integration
    memory_integration_config_t config = {
        .enable_leak_detection = true,
        .enable_use_after_free_detection = true,
        .enable_double_free_detection = true,
        .track_allocations = true,
        .max_tracked_allocations = 10000,
        .memory_subsystem_path = NULL
    };

    // Initialize
    memory_integration_context_t* ctx =
        memory_integration_initialize(&config);
    if (!ctx) {
        fprintf(stderr, "Failed to initialize Memory Integration\n");
        return 1;
    }

    // Detect memory leaks
    leak_detection_report_t report = {0};
    report.max_records = 100;

    crrss_status_t status = memory_integration_detect_leaks(ctx, &report);

    if (status == CRRSS_SUCCESS) {
        printf("Memory Leak Analysis:\n");
        printf("  Potential Leaks: %u\n", report.potential_leaks);
        printf("  Total Leaked: %lu bytes\n", report.total_leaked_bytes);

        if (report.leak_records) {
            printf("\nLeak Details:\n");
            for (uint32_t i = 0; i < report.potential_leaks &&
                i < report.max_records; i++) {
                printf("  %s: %zu bytes\n",
                    report.leak_records[i].allocation_site,
                    report.leak_records[i].size);
            }
            free(report.leak_records);
        }
    }

    // Cleanup
    memory_integration_shutdown(ctx);
    return 0;
}
```

# Configuration

## Environment Variables

```
# Set CRRSS data directory
export CRRSS_DATA_DIR=/path/to/crrss/data

# Enable debug output
export CRRSS_DEBUG=1

# Set log level
export CRRSS_LOG_LEVEL=info # debug, info, warning, error
```

## Configuration Files

CRRSS can be configured via JSON configuration files:

```
{
  "bpme": {
    "enable_pattern_matching": true,
    "confidence_threshold": 0.7,
    "max_predictions": 1000
  },
  "sciv": {
    "enable_strict_mode": false,
    "coding_standard": "kernel",
    "max_function_complexity": 20
  },
  "memory": {
    "enable_leak_detection": true,
    "track_allocations": true,
    "max_tracked_allocations": 10000
  }
}
```

Load configuration:

```
crrss --config crrss_config.json query --file myfile.c
```

## Best Practices

### 1. Regular Scans

Run CRRSS regularly on your codebase:

```
# Daily scan
crrss query --priority P0 --priority P1

# Weekly full analysis
crrss stats --directory . --memory
```

## 2. Integration with Development Workflow

```
# Pre-commit hook
#!/bin/bash
FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(c|h)$')
for FILE in $FILES; do
    crrss query --file "$FILE" --priority P0 --priority P1
    if [ $? -ne 0 ]; then
        echo "CRRSS found critical issues in $FILE"
        exit 1
    fi
done
```

## 3. CI/CD Integration

```
# GitHub Actions example
- name: Run CRRSS Analysis
  run: |
    cd tools/crrss && mkdir build && cd build
    cmake ..
    make
    ./crrss stats --directory ../../moduler_kernel --format json > crrss_report.json
```

## 4. Focused Analysis

For large codebases, focus on high-risk areas:

```
# Analyze memory management
crrss query --category memory --directory moduler_kernel/master_memory_manager/

# Analyze concurrency code
crrss query --category concurrency --directory moduler_kernel/orchestrator/
```

# Troubleshooting

## Common Issues

### 1. CRRSS Not Found

**Problem:** Command not found after building

**Solution:**

```
# Use from build directory
cd tools/crrss/build
./crrss help

# Or add to PATH
export PATH=$PATH:/path/to/crrss/build
```

### 2. Initialization Failed

**Problem:** CRRSS fails to initialize

**Solution:**



```
# Check for required libraries
ldd ./crrss

# Ensure all dependencies are installed
sudo apt-get install build-essential cmake
```

### 3. Too Many False Positives

**Problem:** CRRSS reports too many false positives

**Solution:**

- Increase confidence threshold in configuration
- Use priority filtering: `--priority P0 --priority P1`
- Focus on specific categories

### 4. File Not Found

**Problem:** CRRSS can't find files to analyze

**Solution:**

```
# Use absolute paths
crrss query --file /full/path/to/file.c

# Or run from correct directory
cd /path/to/project
crrss query --file relative/path/to/file.c
```

## Debug Mode

Enable debug output for troubleshooting:

```
# Set debug environment variable
export CRRSS_DEBUG=1
crrss query --file myfile.c

# Check configuration
crrss version # Shows build configuration
```

## Performance Issues

If CRRSS is slow:

1. **Reduce scope:** Analyze specific files instead of entire directories
2. **Disable features:** Turn off unused validation rules
3. **Increase thresholds:** Higher confidence thresholds = fewer results = faster

```
// Configure for performance
sciv_config_t config = {
    .enable_strict_mode = false,
    .enable_style_checks = false, // Disable if not needed
    .enable_performance_checks = false, // Disable if not needed
    // ...
};
```

## Getting Help

---

### Built-in Help

```
# General help
crrss help

# Command-specific help
crrss query --help
crrss stats --help
```

### Documentation

- Main README: `/tools/crrss/README.md`
- Architecture docs: `/tools/crrss/docs/ARCHITECTURE.md`
- API reference: `/tools/crrss/docs/API_REFERENCE.md`

### Support

- GitHub Issues: Report bugs and request features
- Code examples: `/tools/crrss/examples/`
- Test suite: `/tools/crrss/tests/` (examples of usage)

## Examples Repository

---

See `/tools/crrss/examples/` for complete working examples of:

- File analysis
- Directory scanning
- Memory leak detection
- Custom pattern detection
- Report generation
- CI/CD integration

## Summary

---

CRRSS provides powerful tools for ensuring code quality in the BDI project. Key points:

- Use `query` for finding specific issues
- Use `stats` for overall codebase health
- Integrate into development workflow for continuous monitoring
- Configure thresholds and rules based on your needs
- Focus on high-priority issues first (P0, P1)

For more information, consult the main README and API documentation.